

[← Go Back](#)

Hardware

GIGA Display Shield



Tutorials

[Getting Started with the GIGA Display Shield](#)[GIGA Display Shield GFX Guide](#)[GIGA Display Shield LVGL Guide](#)[GIGA Display Shield Draw Images Guide](#)[GIGA Display Shield Touch Interface](#)[GIGA Display Shield Automatic Screen Orientation with IMU](#)[GIGA Display Shield Microphone Guide](#)[GIGA Display Shield Camera Connector Guide](#)[GIGA Display Shield 3D Engine Tutorial](#)[SquareLine Studio With the GIGA Display Shield](#)[Display Camera Feed on GIGA Display Shield with OpenMV](#)[GIGA Display Shield emWin Guide](#)[GIGA Display Shield AppWizard Guide](#)[GIGA Display Shield Display Image From USB](#)

[Home](#) / [Hardware](#) / [GIGA Display Shield](#) / [GIGA Display Shield Display Image From USB](#)

GIGA Display Shield Display Image From USB

Learn to display RAW Images on the GIGA Display from USB Storage.

Pedro
Author · Sousa Lima · Last revision · 03/10/2025

ON THIS PAGE

Introduction

[Hardware Requirements](#)[Software Requirements](#)[Features](#)[Preparing Your Image](#)[Code Breakdown](#) +

Introduction

This guide explores how to read and display raw RGB565 images from a USB drive onto an Arduino GIGA Display Shield. Making it possible for you to list files from a USB drive, select an image, and render it.



Display with image from USB

Hardware Requirements

[Arduino GIGA R1 WiFi](#)[Arduino GIGA Display](#)[Help](#)

USB mass storage device
(formatted with FAT32)

Software

Requirements

USBHostMbed5 (for USB host functionality)

FATFileSystem (to read files from USB)

ArduinoH7Video (Included in the core) and

ArduinoGraphics

libraries (for handling display rendering)

Features

Lists all files in the `/usb/` directory.

Allows users to select an image via the Serial Monitor.

Automatically reads and displays an **800x480 RGB565** image.

Clears the display upon request.

Preparing Your Image

To ensure your image is in the correct format (800x480, 16-bit RGB565), you can use an online converter such as the **LVGL Image Converter**. Select **RGB565** as the output format and set the resolution to 800x480 before saving the file

FAT32. For testing we used the image:



Unprepared test image

For image conversion we selected the following settings:

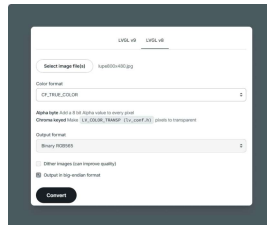


Image settings

Code Breakdown

Setting Up USB Host and Display

With the following code the USB host is initialized, enabling support for mass storage devices. The display is also set up using the

`Arduino_H7_Video` class.

```
1 USBHostMSD msd;  
2 mbed::FATFileSystem us  
3 Arduino_H7_Video Displ
```

Listing Files on USB

```

1 void listRootDirector
2     fileCount = 0;
3     DIR* dir = opendir(
4     if (!dir) return;
5     while (true) {
6         struct dirent* en
7         if (!entry || fil
8         strncpy(fileNames
9         fileCount++;
10    }
11    closedir(dir);
12 }

```

Now by entering a file number in the Serial Monitor an image is selected and will be displayed on the GIGA Display Shield.

[illegible]

Reading and Displaying RAW Images

The file is read row-by-row (800 pixels per row, 2 bytes per pixel). If the entire image (800x480 pixels) were loaded into memory at once, it would require approximately 768 KB of RAM ($800 * 480 * 2$ bytes). However, by processing only one row at a time ($800 * 2$ bytes), the memory usage is reduced to just `1.6 KB`, making this approach much more efficient and feasible for devices with restricted memory (like the GIGA). This approach minimizes memory usage by processing smaller chunks of the image at one time, avoiding large allocations that may exceed available RAM (512 KB for the GIGA). We use, `fopen()` and `fclose()` are used for file management. `fopen()` allows us to open the image file and read data as needed, rather than loading everything at once. `fclose()` ensures that the file is properly closed after reading, freeing up system resources and preventing potential memory leaks.

```
1  bool displayRawRowByR
2  FILE* f = fopen(pat
3  if (!f) return fals
4
5  uint8_t* rowBuffer
6  if (!rowBuffer) {
7      fclose(f);
8      return false;
9  }
10
11  Display.beginDraw()
12  for (int y = 0; y <
13      fread(rowBuffer,
14      Image rowImage(EN
15      Display.image(row
16  }
17  Display.endDraw();
18
19  free(rowBuffer);
20  fclose(f);
21  return true;
22 }
```

Clearing the Display

Users can reset the display by entering `clear` in the Serial Monitor. This simply fills all pixels with black.

```
1  void forceScreenClear(
2      Display.beginDraw();
3      Display.fill(0x0000)
4      Display.endDraw();
5  }
```

Full Code

```
1  #include <Arduino.h>
2  #include <DigitalIO.h>
3  #include <FATFileSyst
4  #include <Arduino.h>
5  #include <ArduinoG
6
7  USBHostMSD msd;
8  mbed::FATFileSyst
9
10 const int USB_HOST_
11 const int MAX_FILE$
12 const int MAX_NAME_
13 char fileNames[MAX_
14 int fileCount = 0;
15 const int MAX_CONNE
16
17 Arduino_H7_Video Di
18 const int IMG_WIDTH
19 const int IMG_HEIGH
20 const uint32_t EXPE
21
22 uint8_t rowBuffer[1
23
24 void setup() {
25     Serial.begin(11
26     pinMode(USB_HOS
27     digitalWrite(US
28     while (!Serial)
```

Conclusion

This project demonstrates how to read and display **16-bit RGB565 images** from a USB drive onto the Arduino GIGA Display. The row-by-row approach optimizes memory usage while ensuring smooth rendering.

Suggest changes **Need support?**

The content on [docs.arduino](#) [Help Center](#) [Ask the Arduino](#)

License

The Arduino documentation is licensed under the

public
GitHub
repository.
If you see
anything
wrong, you
can edit
this page
here.

Discord

[Share Alike](#)
[4.0](#) license.

Was this article helpful?

